

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

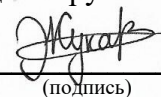
«Обеспечение информационной безопасности мобильных устройств»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Создание приложения и поиск URL, конечных точек и секретов в APK-файлах (Kotlin)»

Выполнили:

Жук Анастасия Валерьевна,
студент группы N3348


(подпись)

Проверил:

Федоров Иван Романович,
научный сотрудник, доцент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

Введение	3
1 Разработка приложения в Android Studio	4
2 Получение арк. файла.....	21
2.1 Без обфускации	21
2.2 С обфускацией	22
3 Декомпиляция арк. файлов с помощью ArkTool	24
3.1 Без обфускации	24
3.2 С обфускацией	24
3.3 Сравнение.....	25
4 Сканирование приложения утилитой arkleaks	26
5 Сканирование стороннего приложения (.apk) утилитой arkleaks	29
5.1 Сравнение с разработанным приложением.....	29
6 Проверка разработанное приложение через VirusTotal	31
Заключение.....	33

ВВЕДЕНИЕ

Цель работы – освоить процессы разработки Android-приложений на Kotlin и проведения анализа безопасности APK-файлов, включая их декомпиляцию, поиск уязвимостей (URL, конечных точек и секретов) с помощью специализированных инструментов (ApkTool, apkleaks, VirusTotal), а также оценить эффективность обфускации кода.

Для достижения поставленной цели необходимо решить следующие задачи:

- в Android Studio создать проект мобильного приложения на Kotlin, состоящее из трех экранов: экран с выводом ФИО студента, экран, отображающий данные, полученные через вызов к REST API (можно использовать любой доступный публичный REST API), и экран с формой обратной связи (можно реализовать взаимодействие с базой данных);
- получить apk-файл двумя способами: с обфускацией и без обфускации;
- декомпилировать полученные apk с помощью ApkTool, сравнить результаты;
- просканировать разработанное приложение утилитой apkleaks;
- просканировать любое стороннее приложение (.apk) утилитой apkleaks и сравнить результат;
- проверить разработанное приложение через VirusTotal.

1 РАЗРАБОТКА ПРИЛОЖЕНИЯ В ANDROID STUDIO

Создадим проект. Для этого открываем Android Studio, выбираем шаблон «Empty Views Activity» (см. рисунок 1), нажимаем кнопку «Next», даем название проекту (я использовала название по умолчанию), выбираем локацию проекта, язык программирования Kotlin (условие работы, но он выбирается по умолчанию) и минимальный SDK такой, чтобы процент дивайсов, на которых можно будет запустить приложение, был как можно больше (см. рисунок 2). Нажимаем кнопку «Finish».

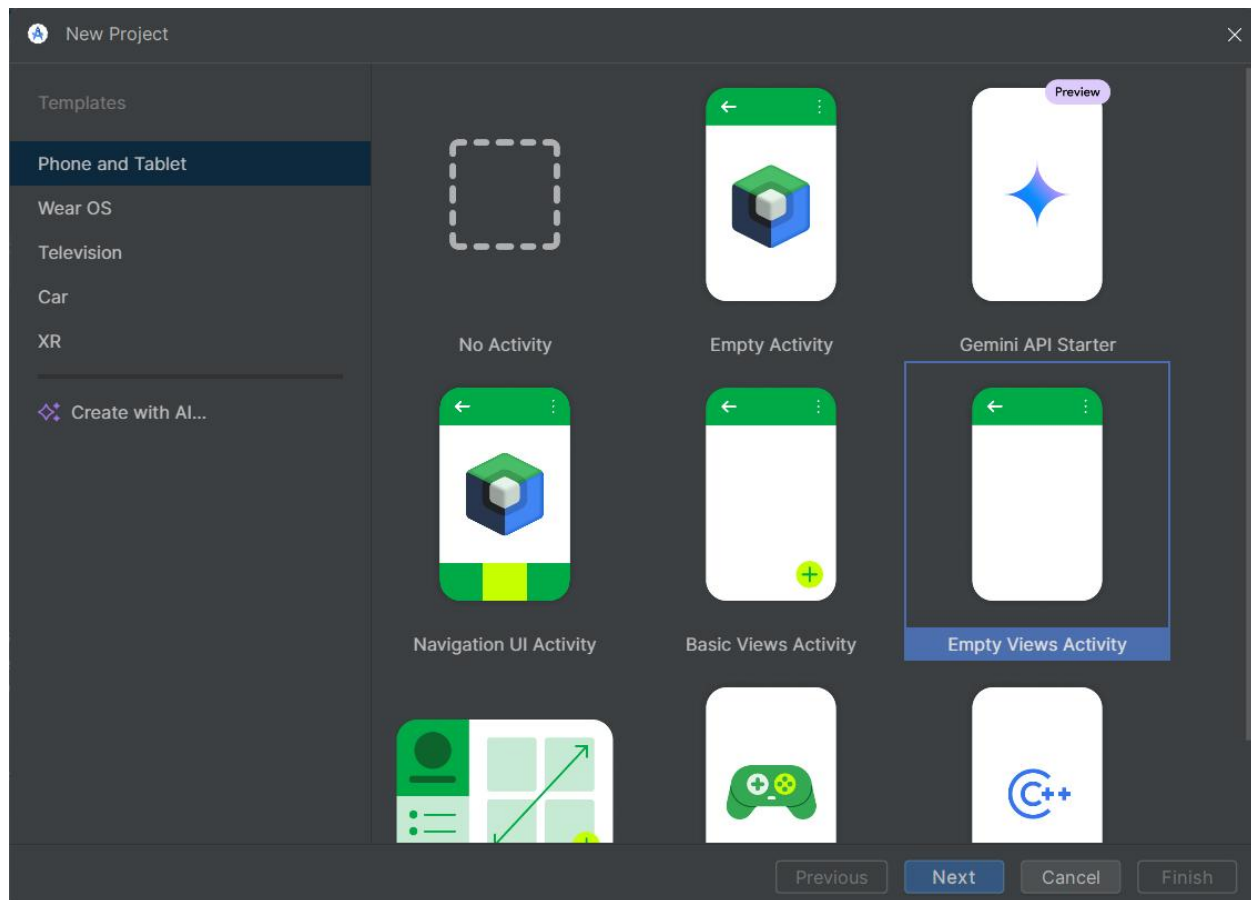


Рисунок 1 – Выбор шаблона

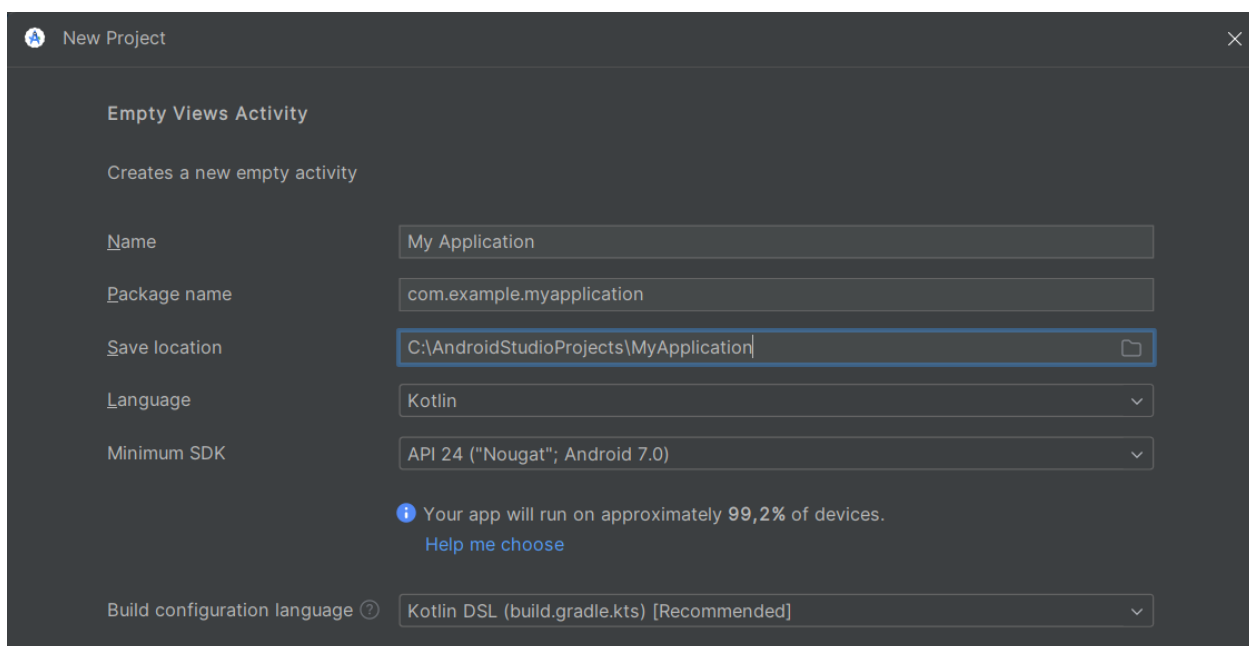


Рисунок 2 – Создание нового проекта

После этого можно настроить сам дивайс, на котором будет тестироваться наше приложение в будущем. Есть два варианта: виртуальный и физический. Я пробовала сделать виртуальный, но у меня он не запускался, поэтому я воспользовалась своим смартфоном Xiaomi Redmi 9T. Чтобы его добавить в Android Studio, необходимо на самом смартфоне включить режим разработчика и дать некоторые разрешения. Чтобы включить режим разработчика на Redmi 9T, нужно 7 раз нажать на версию MIUI в настройках. Открываем «Настройки», затем «О телефоне», нажимаем на «Версию MIUI» 7-8 раз, пока не появится сообщение «Теперь вы – разработчик». После этого в «Расширенных настройках» появятся дополнительные опции. Там открываем «Для разработчиков» и делаем активными правила «Отладка по USB», «Установка через USB». Чтобы сделать активной «Беспроводную отладку», что также необходимо для успешной установки приложения на смартфон из Android Studio, необходимо перейти в этот раздел, включить «Беспроводная отладка» и нажать «Подключение устройств через QR-код». В Android Studio находим выпадающий список с устройствами, а там нажимаем на «Pair Devices Using Wi-Fi» (см. рисунок 3). Появится QR-код. Его нужно будет отсканировать телефоном и подождать подключения. Важно, что телефон должен быть подключен к компьютеру по USB-кабелю. Теперь можно переходить к написанию приложения.

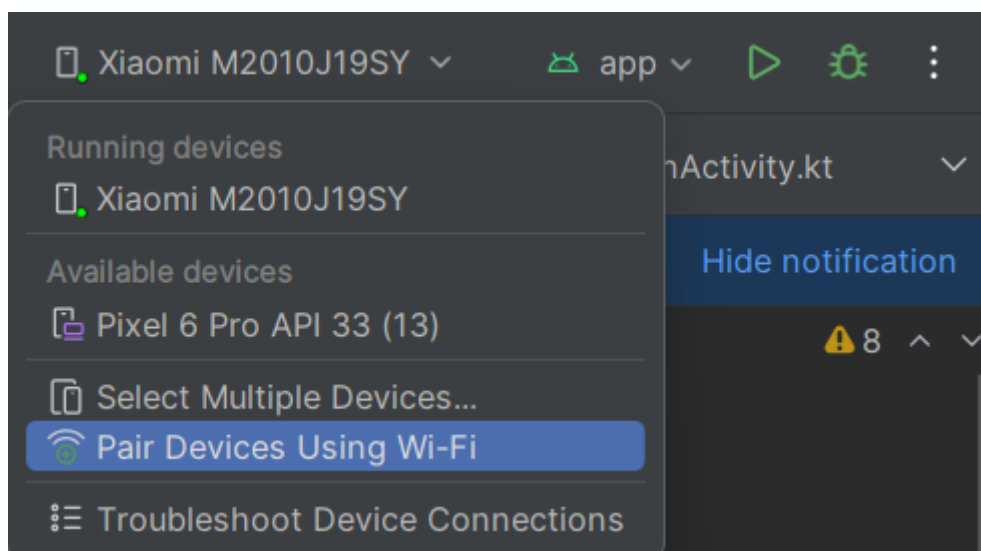


Рисунок 3 – Добавление устройства

На листингах 1 – 8 будут приведены обновленные и созданные файлы, на рисунке 4 представлена структура проекта. Единственное, что необходимо отметить, что после того, как буду внесены изменения в файл `build.gradle.kts`, нужно нажать «Sync Now» на появившейся сверху синей полосе.

Листинг 1 – Файл `build.gradle.kts` (путь `..\build.gradle.kts`)

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace = "com.example.myapplication"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.example.myapplication"
        minSdk = 24
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}
```

```

}

dependencies {
    implementation(libs.androidx.activity.ktx)
    implementation(libs.androidx.appcompat)
    implementation(libs.androidx.constraintlayout)
    implementation(libs.androidx.core.ktx)
    implementation(libs.material)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(libs.androidx.junit)

    // Retrofit для API
    implementation("com.squareup.retrofit2:retrofit:2.9.0")
    implementation("com.squareup.retrofit2:converter-gson:2.9.0")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
}

```

Листинг 2 – Файл AndroidManifest.xml (путь ..\app\src\main\AndroidManifest.xml)

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication"
        tools:targetApi="31">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity android:name=".ApiActivity" />
        <activity android:name=".FeedbackActivity" />

    </application>
</manifest>

```

Листинг 3 – Файл activity_main.xml (путь ..\app\src\main\res\layout\activity_main.xml)

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView

```

```

        android:id="@+id/tvStudentName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Жук Анастасия"
        android:textSize="24sp"
        android:textStyle="bold"
        android:layout_marginBottom="32dp"/>

        <Button
            android:id="@+id/btnToApi"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Перейти к API"
            android:layout_marginBottom="12dp"/>

        <Button
            android:id="@+id/btnToFeedback"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Форма обратной связи"/>

    </LinearLayout>

```

Листинг 4 – Файл MainActivity.kt (путь ..\app\src\main\java\com\example.myapplication\MainActivity.kt)

```

package com.example.myapplication

import android.content.Intent
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnToApi = findViewById<Button>(R.id.btnToApi)
        val btnToFeedback = findViewById<Button>(R.id.btnToFeedback)

        btnToApi.setOnClickListener {
            startActivity(Intent(this, ApiActivity::class.java))
        }

        btnToFeedback.setOnClickListener {
            startActivity(Intent(this, FeedbackActivity::class.java))
        }
    }
}

```

Листинг 5 – Файл activity_api.xml (путь ..\app\src\main\res\layout\activity_api.xml)

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView

```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Данные из API"
        android:textSize="20sp"
        android:textStyle="bold"
        android:layout_marginBottom="16dp"/>

<Button
    android:id="@+id/btnLoadData"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Загрузить данные"
    android:layout_marginBottom="16dp"/>

<TextView
    android:id="@+id/tvApiResponse"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Нажмите кнопку для загрузки"
    android:background="#F0F0F0"
    android:padding="12dp"/>

</LinearLayout>

```

Листинг 6 – Файл ApiActivity.kt (путь ..\app\src\main\java\com\example.myapplication\ApiActivity.kt)

```

package com.example.myapplication

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import retrofit2.http.GET

// Интерфейс API
interface JsonPlaceholderApi {
    @GET("posts/1")
    suspend fun getPost(): Post
}

// Модель данных
data class Post(
    val userId: Int,
    val id: Int,
    val title: String,
    val body: String
)

class ApiActivity : AppCompatActivity() {
    private lateinit var tvApiResponse: TextView
    private lateinit var btnLoadData: Button

    override fun onCreate(savedInstanceState: Bundle?) {

```

```

super.onCreate(savedInstanceState)
setContentView(R.layout.activity_api)

tvApiResponse = findViewById(R.id.tvApiResponse)
btnLoadData = findViewById(R.id.btnLoadData)

btnLoadData.setOnClickListener {
    loadDataFromApi()
}

private fun loadDataFromApi() {
    val retrofit = Retrofit.Builder()
        .baseUrl("https://jsonplaceholder.typicode.com/")
        .addConverterFactory(GsonConverterFactory.create())
        .build()

    val api = retrofit.create(JsonPlaceholderApi::class.java)

    CoroutineScope(Dispatchers.IO).launch {
        try {
            val post = api.getPost()
            withContext(Dispatchers.Main) {
                tvApiResponse.text = ""
                ID: ${post.id}
                User ID: ${post.userId}
                Title: ${post.title}
                Body: ${post.body}
                """".trimIndent()
            }
        } catch (e: Exception) {
            withContext(Dispatchers.Main) {
                Toast.makeText(this@ApiActivity, "Ошибка: ${e.message}",
                    Toast.LENGTH_SHORT).show()
            }
        }
    }
}
}

```

**Листинг 7 – Файл activity_feedback.xml (путь
 ..\app\src\main\res\layout\activity_feedback.xml)**

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Форма обратной связи"
        android:textSize="20sp"
        android:textStyle="bold"
        android:layout_marginBottom="16dp"/>

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"

```

```

        android:layout_height="wrap_content"
        android:hint="Ваше имя"
        android:layout_marginBottom="12dp"/>

<EditText
    android:id="@+id/etMessage"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Сообщение"
    android:minLines="3"
    android:gravity="top"
    android:layout_marginBottom="16dp"/>

<Button
    android:id="@+id/btnSend"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Отправить"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Сохранённые сообщения:"
    android:textStyle="bold"
    android:layout_marginTop="24dp"
    android:layout_marginBottom="8dp"/>

<TextView
    android:id="@+id/tvMessagesList"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="#F0F0F0"
    android:padding="12dp"/>

</LinearLayout>

```

Листинг 8 – Файл FeedbackActivity.kt (путь

..\app\src\main\java\com\example.myapplication\FeedbackActivity.kt)

```

package com.example.myapplication

import android.content.Context
import android.content.ContentValues
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast

class FeedbackDbHelper(context: Context) : SQLiteOpenHelper(context,
    "feedback.db", null, 1) {

    companion object {
        private const val TABLE_NAME = "messages"
        private const val COL_ID = "id"
        private const val COL_NAME = "name"
        private const val COL_MESSAGE = "message"
    }
}

```

```

        override fun onCreate(db: SQLiteDatabase) {
            db.execSQL("""
                CREATE TABLE $TABLE_NAME (
                    $COL_ID INTEGER PRIMARY KEY AUTOINCREMENT,
                    $COL_NAME TEXT NOT NULL,
                    $COL_MESSAGE TEXT NOT NULL
                )
            """)
        }

        override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion:
Int) {
            db.execSQL("DROP TABLE IF EXISTS $TABLE_NAME")
            onCreate(db)
        }

        fun insertMessage(name: String, message: String) {
            val db = writableDatabase
            val values = ContentValues().apply {
                put(COL_NAME, name)
                put(COL_MESSAGE, message)
            }
            db.insert(TABLE_NAME, null, values)
            db.close()
        }

        fun getAllMessages(): List<String> {
            val messages = mutableListOf<String>()
            val db = readableDatabase
            val cursor = db.rawQuery("SELECT * FROM $TABLE_NAME ORDER BY id
DESC", null)

            if (cursor.moveToFirst()) {
                do {
                    val name =
cursor.getString(cursor.getColumnIndexOrThrow(COL_NAME))
                    val msg =
cursor.getString(cursor.getColumnIndexOrThrow(COL_MESSAGE))
                    messages.add("Имя: $name\nСообщение: $msg")
                } while (cursor.moveToNext())
            }
            cursor.close()
            db.close()
            return messages
        }
    }

    class FeedbackActivity : AppCompatActivity() {
        private lateinit var etName: EditText
        private lateinit var etMessage: EditText
        private lateinit var btnSend: Button
        private lateinit var tvMessagesList: TextView
        private lateinit var dbHelper: FeedbackDbHelper

        override fun onCreate(savedInstanceState: Bundle?) {
            super.onCreate(savedInstanceState)
            setContentView(R.layout.activity_feedback)

            etName = findViewById(R.id.etName)
            etMessage = findViewById(R.id.etMessage)
            btnSend = findViewById(R.id.btnSend)

```

```

        tvMessagesList = findViewById(R.id.tvMessagesList)
        dbHelper = FeedbackDbHelper(this)

        btnSend.setOnClickListener { saveFeedback() }
        loadMessages()
    }

    private fun saveFeedback() {
        val name = etName.text.toString().trim()
        val message = etMessage.text.toString().trim()

        if (name.isEmpty() || message.isEmpty()) {
            Toast.makeText(this, "Заполните все поля",
                Toast.LENGTH_SHORT).show()
            return
        }

        dbHelper.insertMessage(name, message)
        Toast.makeText(this, "Сообщение сохранено",
            Toast.LENGTH_SHORT).show()
        etName.text.clear()
        etMessage.text.clear()
        loadMessages()
    }

    private fun loadMessages() {
        val messages = dbHelper.getAllMessages()
        tvMessagesList.text = if (messages.isEmpty()) "Нет сохранённых
сообщений"
        else messages.joinToString("\n\n")
    }

    override fun onDestroy() {
        super.onDestroy()
        dbHelper.close()
    }
}

```

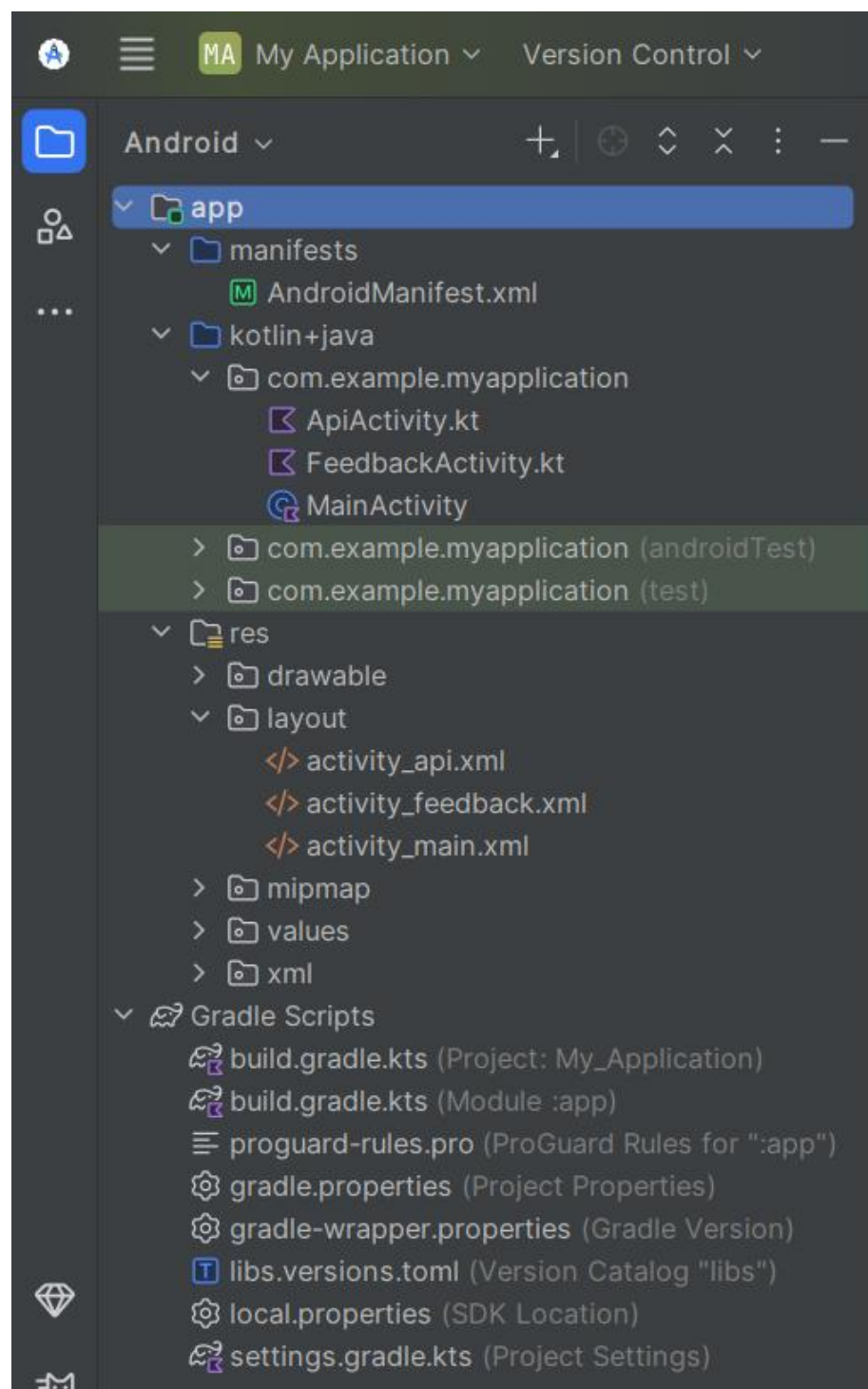


Рисунок 4 – Структура проекта

Для того, чтобы успешно установить приложение на смартфон, необходимо найти и выбрать наше устройство в той вкладке, в который мы искали «Pair Devices Using Wi-Fi». Нажимаем на зеленый значок «Play», и происходит компиляция приложения и его установка на устройство. На рисунке 5 представлена успешная сборка в Android Studio, а на рисунке 6 – сообщение об установке на смартфоне.

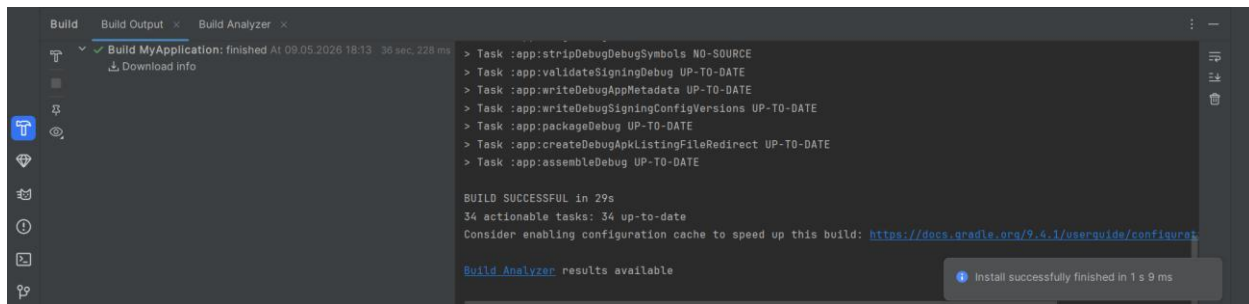


Рисунок 5 – Успешная сборка проекта и установка на смартфон

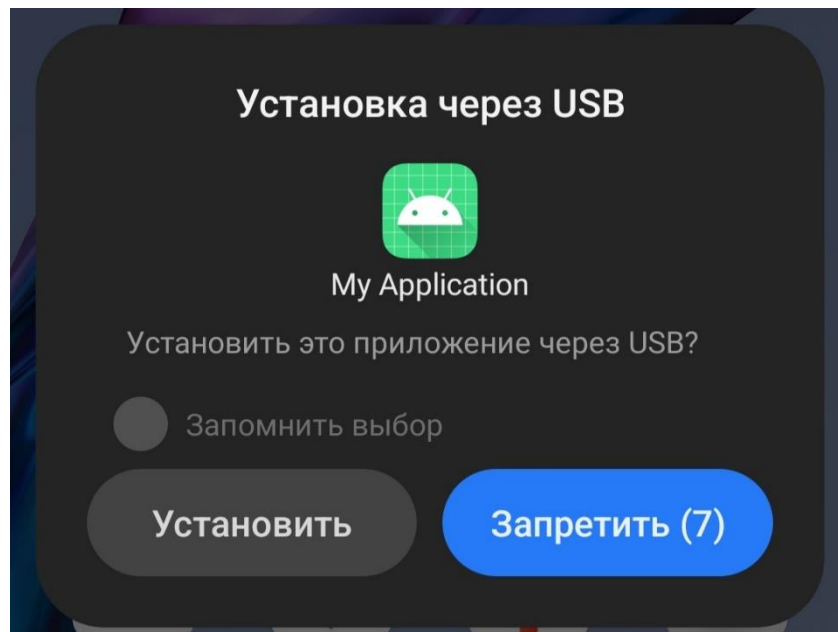


Рисунок 6 – Сообщение об установке приложения на смартфон

На рисунках 7 – 11 представлено приложение и его работа на смартфоне.

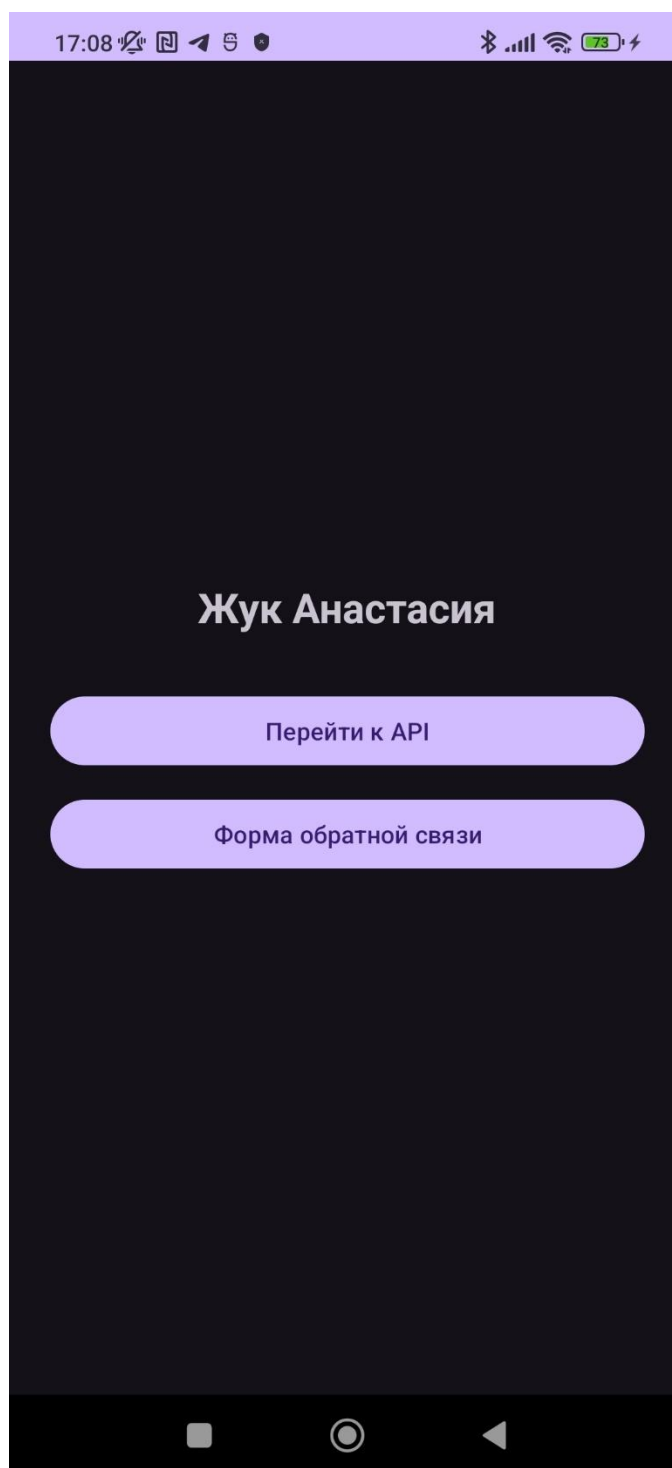


Рисунок 7 – Главный экран с ФИО студента

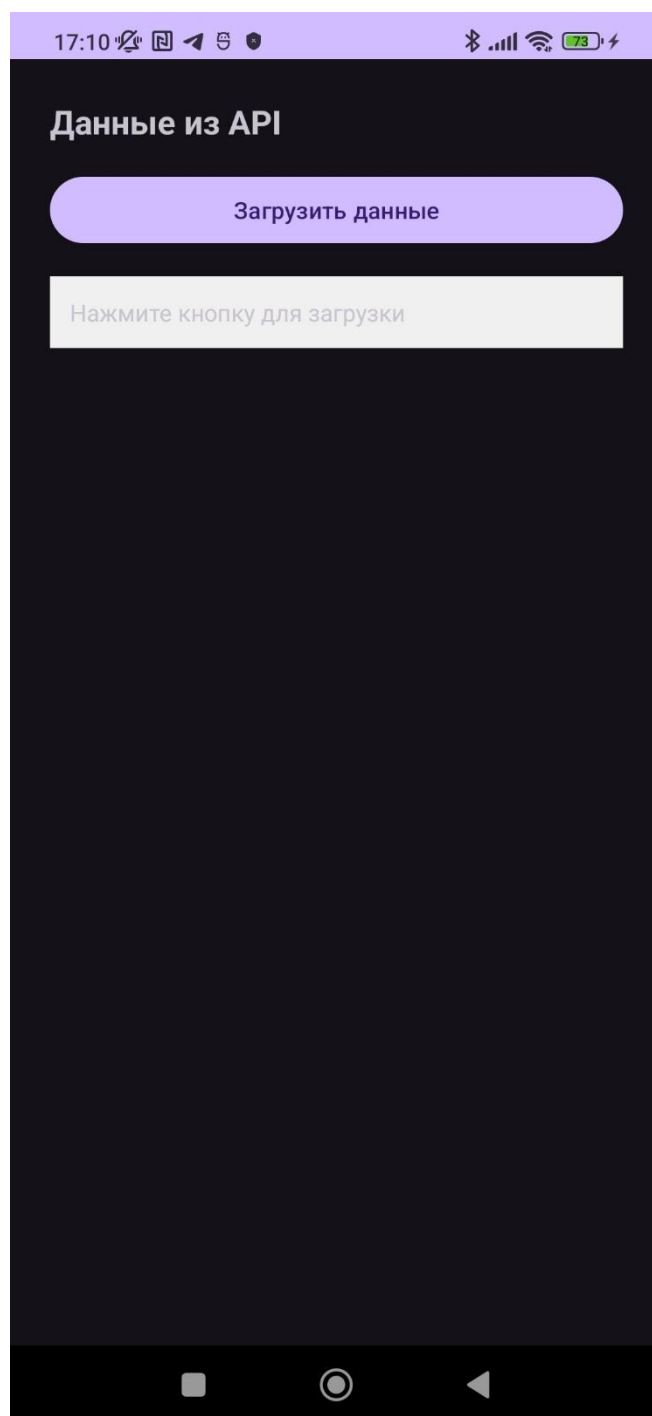


Рисунок 8 – Экран, отображающий данные, полученные через вызов к REST API
(1)

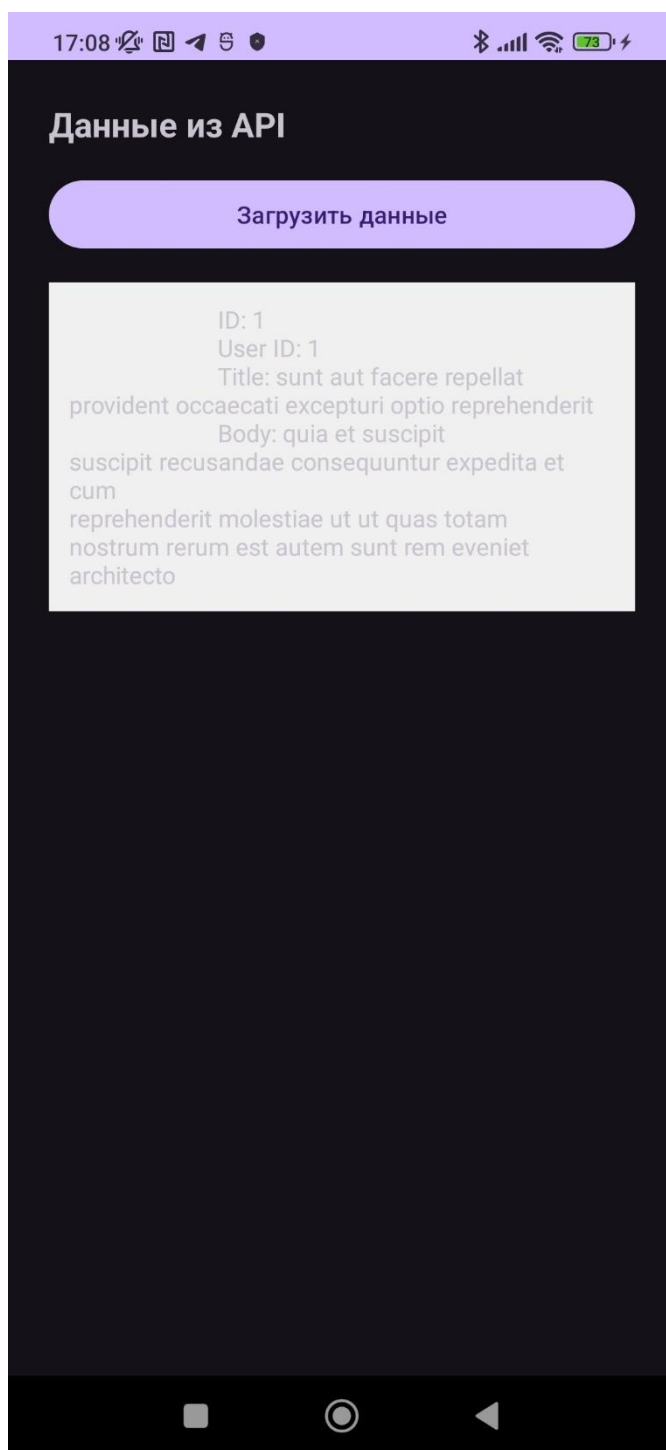


Рисунок 9 – Экран, отображающий данные, полученные через вызов к REST API
(2)

17:08

Форма обратной связи

Ваше имя

Сообщение

Отправить

Сохранённые сообщения:

Нет сохранённых сообщений

Рисунок 10 – Экран с формой обратной связи

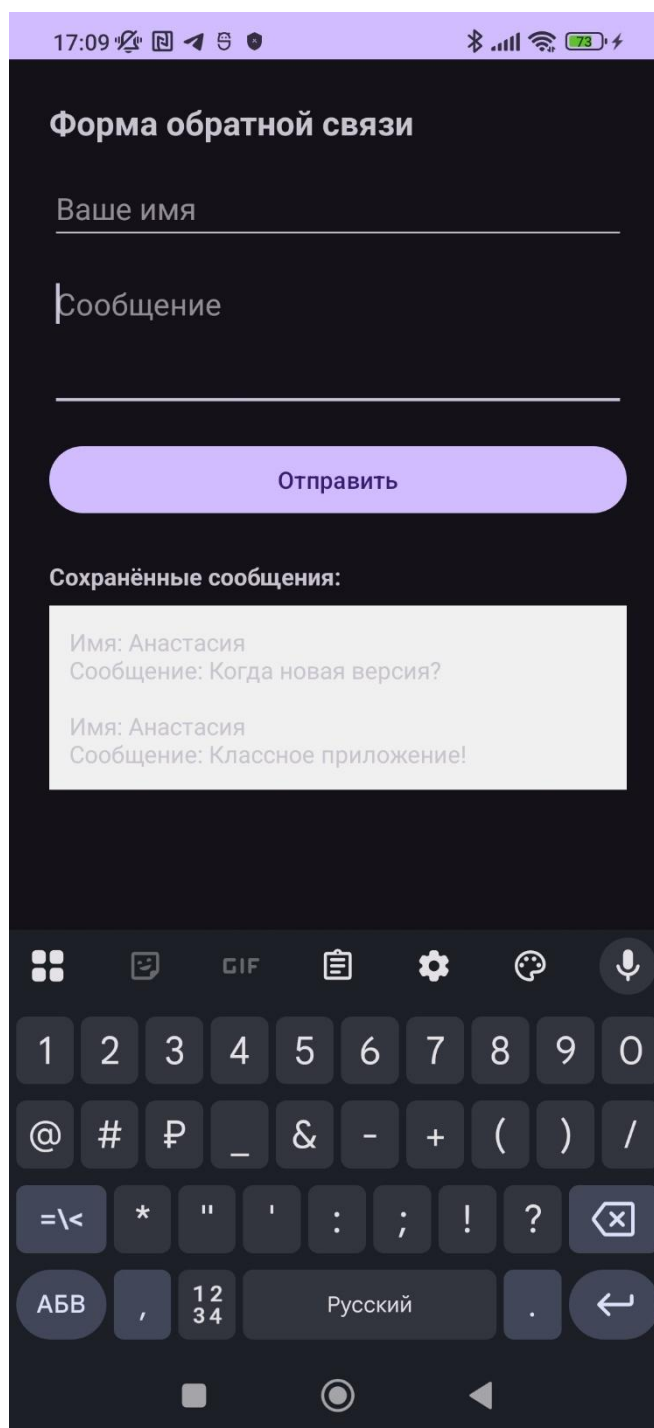


Рисунок 11 – Экран с формой обратной связи. Есть уже несколько комментариев
Вот такое приложение получилось.

2 ПОЛУЧЕНИЕ АРК. ФАЙЛА

Необходимо получить арк. файл двумя способами: с обфускацией и без нее. Перед тем, как переходить к этому шагу работы, необходимо понять, что такое обфускация.

Обфускация (от англ. obfuscate – запутывать, сбивать с толку) – это процесс преобразования исходного кода или данных программы в нечитаемый для человека вид, который при этом сохраняет свою функциональность для компьютера. Это метод защиты интеллектуальной собственности, затрудняющий анализ, декомпиляцию и кражу алгоритмов. Она обеспечивает защиту арк-файла.

2.1 Без обфускации

Для того, чтобы собрать созданное на предыдущем этапе приложение в арк-файл, необходимо в меню Android Studio найти «Build» и нажать на «Assemble Module 'MyApplication.app'» (см. рисунок 12). На рисунке 13 представлен путь к созданному арк-файлу без обфускации.

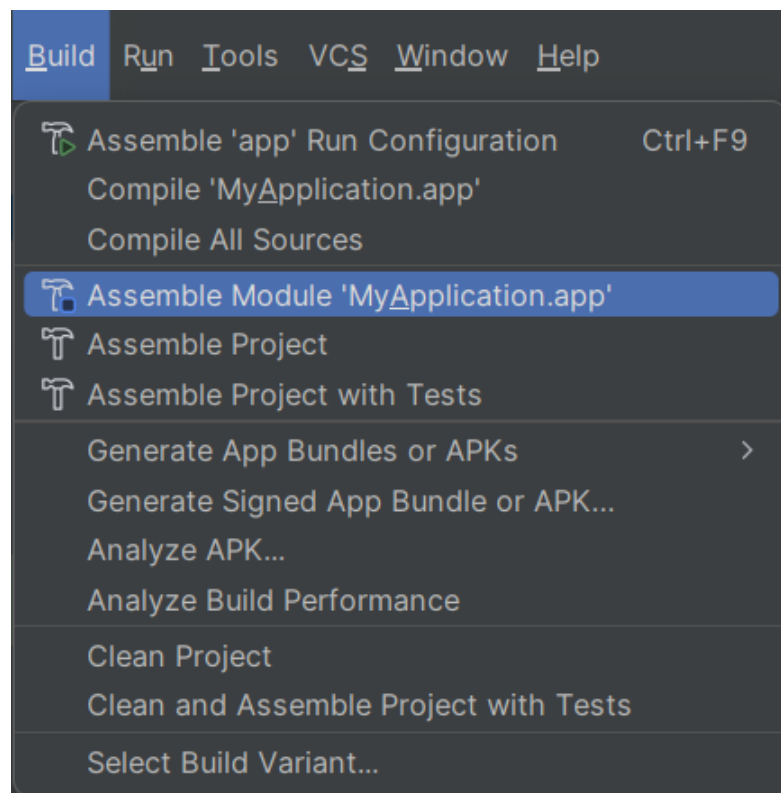


Рисунок 12 – Кнопка для сборки арк-файла

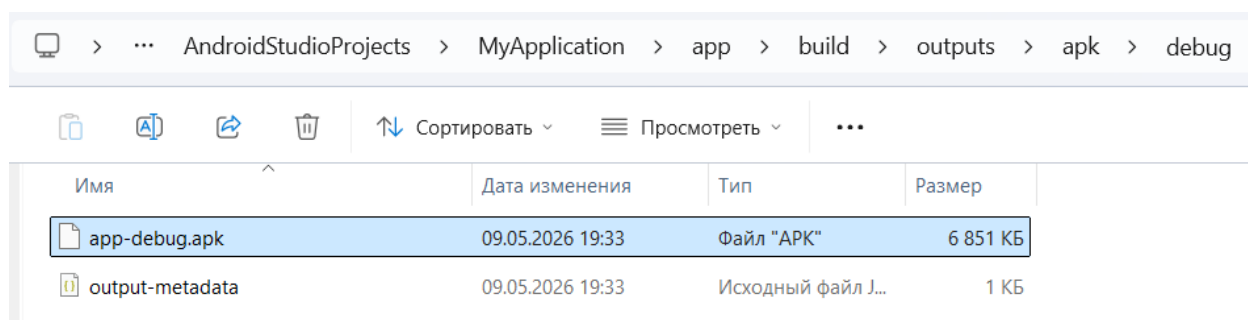


Рисунок 13 – Путь к арк-файлу без обфускации

2.2 С обфускацией

Перед тем, как создавать арк-файл, необходимо добавить некоторые изменения в существующие составляющие приложения. В файле `build.gradle.kts` меняем значение `isMinifyEnabled` на `true` (см. рисунок 14). Нажимаем «Sync Now», а уже после можно собирать все в арк-файл. Также необходимо в меню найти «Build», перейти в «Select Build Variant...» и изменить значение «Active Build Variant» с `debug` на `release` (см. рисунок 15). Снова нажимаем на ту же кнопку, что и на рисунке 12, чтобы скомпилировать арк-файл. На рисунке 16 представлен путь к арк-файлу с обфускацией.

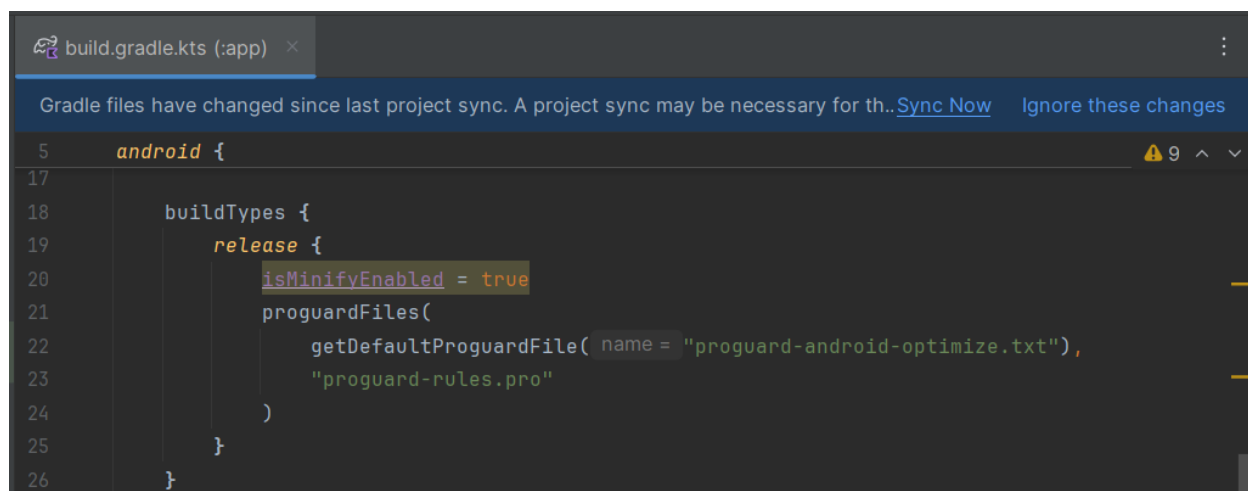


Рисунок 14 – Изменение значения `isMinifyEnabled`

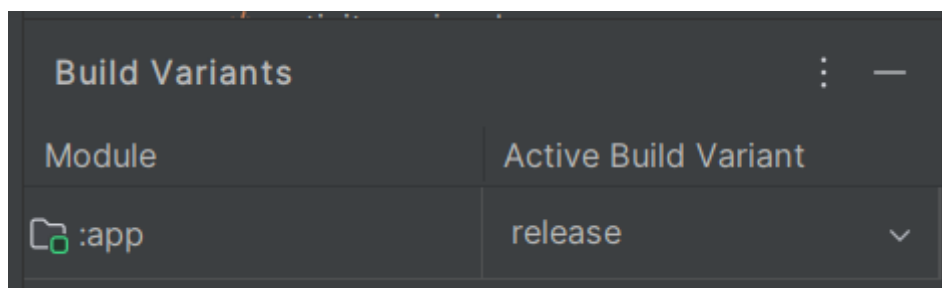


Рисунок 15 – Измененное значение Active Build Variant

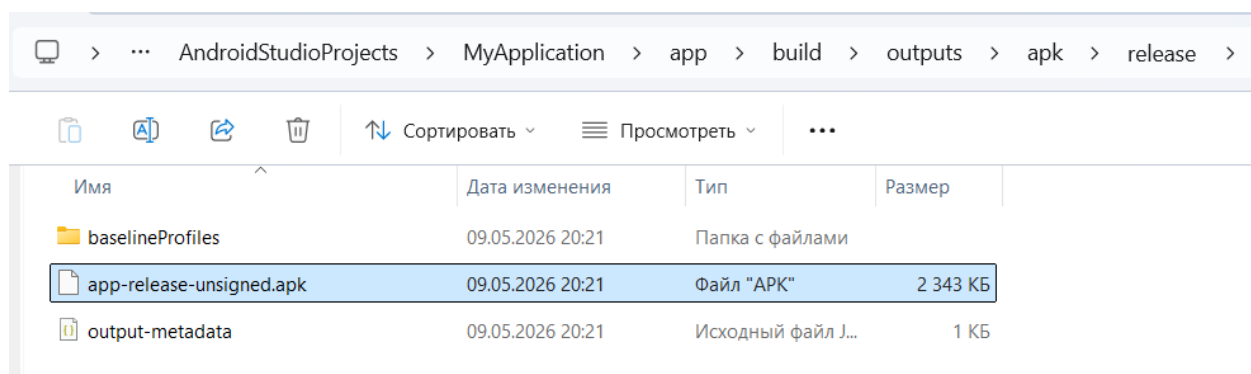


Рисунок 16 – Путь к арк-файлу с обфускацией

3 ДЕКОМПИЛИЯЦИЯ АРК. ФАЙЛОВ С ПОМОЩЬЮ АРКТООЛ

Необходимо скачать ArkTool с официального сайта (инструкция по установке есть на нем). После чего с помощью командной строки от имени администратора запускаем команду `apktool d <apk-path>`.

3.1 Без обфускации

Декомпиляция арк-файла без обфускации представлена на рисунке 17.

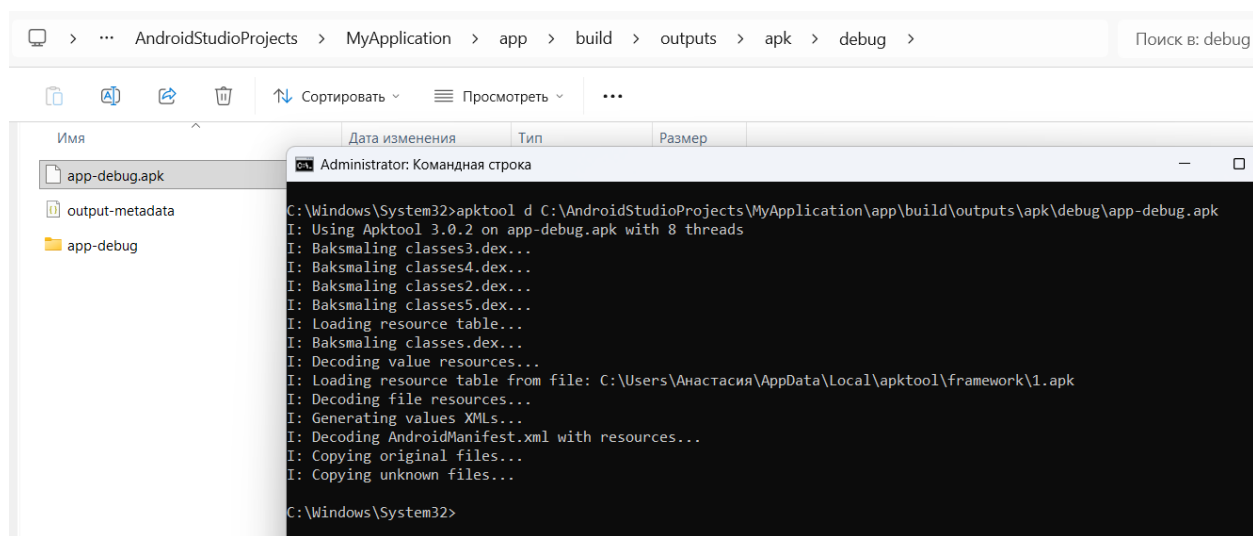


Рисунок 17 – Декомпиляция арк-файла без обфускации

3.2 С обфускацией

Декомпиляция арк-файла с обфускации представлена на рисунке 18.

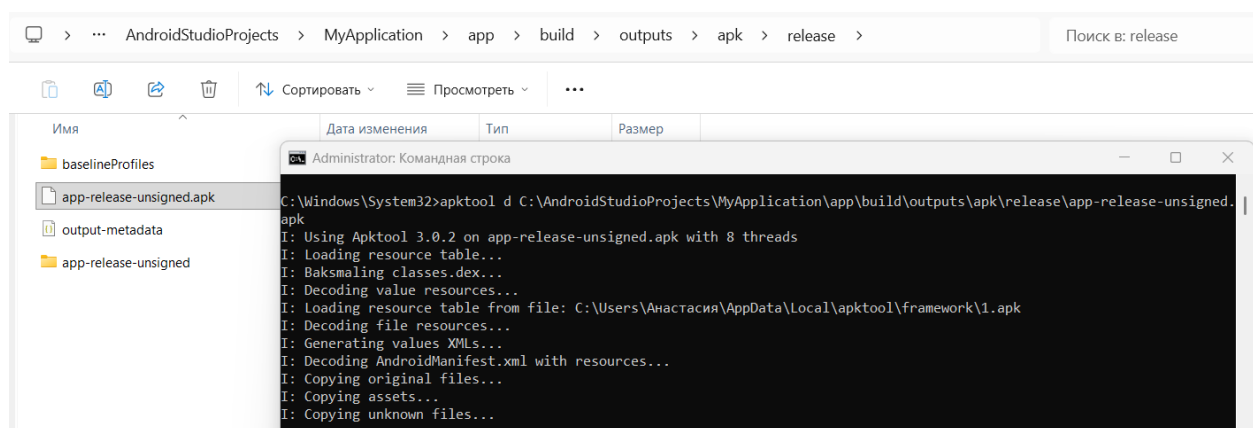


Рисунок 18 – Декомпиляция арк-файлов с обфускацией

3.3 Сравнение

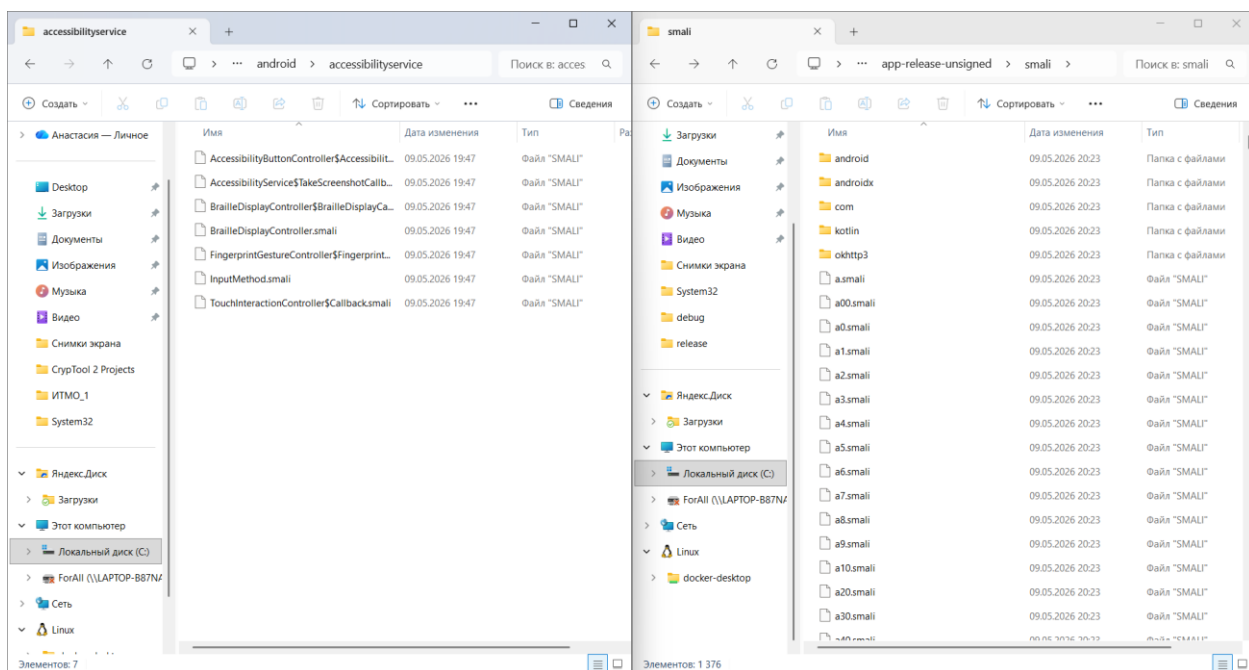


Рисунок 19 – Сравнение содержимого папок с декомпилируемыми арк-файлами (слева без обфускации, справа – с обфускацией). Фрагмент

По умолчанию арк-файлы можно прочитать с помощью декомпилятора и увидеть байт-код, из которого можно многое понять. Когда в файле `build.gradle.kts` мы меняем значение `isMinifyEnabled` на `true`, в работу включаются ProGuard и R8, которые обфусцируют и сокращают код в размере, что в свою очередь затрудняет чтение кода. На рисунке 19 представлены результаты декомпиляции арк-файлов. Результаты декомпиляции я сохраняла в папки (на рисунке представлена лишь часть файлов). Справа видно, что структура стала намного менее читаемая, чем слева. Если посмотреть подробнее, то даже не понятно, в каком пакете что лежит.

4

На Windows это сканирование провести у меня не получилось (пользовательская папка подписана кириллицей), поэтому было принято решение проделать это на Linux Ubuntu. На рисунках 20 – 24 представлены результаты сканирования.

```
admin@Ubuntu:~$ apkleaks -f app-debug.apk -o debug_scan.txt
```



```
v2.6.3
--
Scanning APK file for URIs, endpoints & secrets
(c) 2020-2024, dwisiswant0

Can't find jadx binary.
Do you want to download jadx? (Y/n) y

** Downloading jadx...

** Decompiling APK...
INFO - loading ...
INFO - processing ...
ERROR - finished with errors, count: 20

** Scanning against 'com.example.myapplication'
```

```
[IP_Address]
- 127.0.0.1

[JSON_Web-Token]
- androidGradlePluginVersion=9.2.1
```

Рисунок 20 – Результат сканирования файла app-debug.apk (1)

```

[LinkFinder]
- /...
- /NULL
- /NULL
- /data/misc/profiles/cur/0
- /data/misc/profiles/cur/0/
- /data/misc/profiles/ref/
- /index.html
- /proc/self/fd/
- M/d/yy
- META-INF/services/
- activity_choser_model_history.xml
- dexopt/baseline.prof
- http://schemas.android.com/apk/res-auto
- http://schemas.android.com/apk/res/android
- https://jsonplaceholder.typicode.com/
- share_history.xml

** Results saved into 'debug_scan.txt'.

```

Рисунок 21 – Результат сканирования файла app-debug.apk (2)

```

admin@Ubuntu:~$ cat debug_scan.txt
[IP_Address]
- 127.0.0.1

[JSON_Web_Token]
- androidGradlePluginVersion=9.2.1

[LinkFinder]
- /...
- /NULL
- /NULL
- /data/misc/profiles/cur/0
- /data/misc/profiles/cur/0/
- /data/misc/profiles/ref/
- /index.html
- /proc/self/fd/
- M/d/yy
- META-INF/services/
- activity_choser_model_history.xml
- dexopt/baseline.prof
- http://schemas.android.com/apk/res-auto
- http://schemas.android.com/apk/res/android
- https://jsonplaceholder.typicode.com/
- share_history.xml

```

Рисунок 22 – Содержимое файла debug_scan.txt

```

v2.6.3
--
Scanning APK file for URIs, endpoints & secrets
(c) 2020-2024, dwisiswant0

** Decompiling APK...
INFO - loading ...
INFO - processing ...
ERROR - finished with errors, count: 6

** Scanning against 'com.example.myapplication'

[JSON_Web_Token]
- androidGradlePluginVersion=9.2.1

[LinkFinder]
- /...
- /data/misc/profiles/cur/0/
- /data/misc/profiles/ref/
- /index.html
- dexopt/baseline.prof
- http://schemas.android.com/apk/res-auto
- http://schemas.android.com/apk/res/android

** Results saved into 'release_scan.txt'.

```

Рисунок 23 – Результат сканирования файла app-release-unsigned.apk

```
admin@Ubuntu:~$ cat release_scan.txt
[JSON_Web_Token]
- androidGradlePluginVersion=9.2.1

[LinkFinder]
- /...
- /data/misc/profiles/cur/0/
- /data/misc/profiles/ref/
- /index.html
- dexopt/baseline.prof
- http://schemas.android.com/apk/res-auto
- http://schemas.android.com/apk/res/android
```

Рисунок 24 – Содержимое файла release_scan.txt

5 СКАНИРОВАНИЕ СТОРОННЕГО ПРИЛОЖЕНИЯ (.APK) УТИЛИТОЙ APKLEAKS

В качестве стороннего приложение было выбрано разработанный компанией Disney арк-файл, который уже использовался в предыдущих лабораторных работах. На рисунке 25 представлен результат сканирования.

```
admin@Ubuntu:~$ cat disney_scan.txt
[JSON_Web_Token]
- 1.0.0-alpha02
- androidGradlePluginVersion=8.7.2

[LinkFinder]
- 1b439b897261/446a83a8c549/launch-0d83228f28f3
- https://api.bazaarvoice.com/
- https://api.disneystore.com/facade/facade/api/11000001/
- https://api.shopdisney.com/addrbook/
- https://api.shopdisney.com/andr/api/public/11500001/
- https://api.shopdisney.com/sdsapi/andrprod/
- https://apps.bazaarvoice.com/deployments/DisneyStore/disney_store/production/en_US/container.html
- https://authorization.go.com/
- https://cdn-ssl.img.disneystore.com/content/ds/app/config/
- https://cdn-ssl.img.disneystore.com/content/global/guest_services_sd/
- https://cdn-ssl.s7.shopdisney.com/is/image/ShopDisney/
- https://cdnapisec.kaltura.com/p/2752291/sp/2752291/playManifest/entryId/[videoIdPlaceholder]/format/applehttp/protocol/https/flavorParamIds/301951
- https://en.wikipedia.org/
- https://mct33zhqnxnq6hlpvgdydr9zdl0.device.marketingcloudapis.com/
- https://wallet.wdprapps.disney.com/
- https://wallet.wdprapps.disney.com/api/v1/wallet/
- https://www.disneystore.com
- https://www.disneystore.com/dw/shop/v22_4/
- sizecharts/images/size_diagram_child.jpg
- sizecharts/images/size_diagram_infant.jpg
- sizecharts/images/size_diagram_man.jpg
- sizecharts/images/size_diagram_woman.jpg
- https://fonts.twdc.com/InspireTWDC/latest.css
```

Рисунок 25 – Содержимое файла disney_scan.txt

5.1 Сравнение с разработанным приложением

При сравнении результатов сканирования трех арк-файлов выявлены следующие закономерности. Во-первых, debug-версия разработанного приложения содержит явный URL API (<https://jsonplaceholder.typicode.com/>), что представляет собой потенциальную уязвимость, так как злоумышленник может легко обнаружить endpoint и использовать его для анализа трафика или атак. Во-вторых, release-версия с включенной обфускацией (R8/ProGuard) не показала URL jsonplaceholder.typicode.com в результатах сканирования, что свидетельствует об эффективности механизма обфускации для сокрытия чувствительной информации. Обфускация переименовала классы и методы, а также могла зашифровать или замаскировать строковые литералы. В-третьих, стороннее приложение (Disney Store) продемонстрировало наибольшее количество обнаруженных URL (более 30), что характерно для коммерческих приложений с множеством интеграций. При этом часть

кода также обфусцирована, но строковые литералы с URL остались видимыми, так как необходимы для работы приложения.

Итог: обфускация кода эффективно затрудняет статический анализ арк-файлов и снижает поверхность атаки, скрывая имена классов, методов и потенциально чувствительные данные. Однако полностью устранить утечки информации невозможно, так как некоторые URL и endpoints должны оставаться в открытом виде для функционирования приложения.

6 ПРОВЕРКА РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ ЧЕРЕЗ VIRUSTOTAL

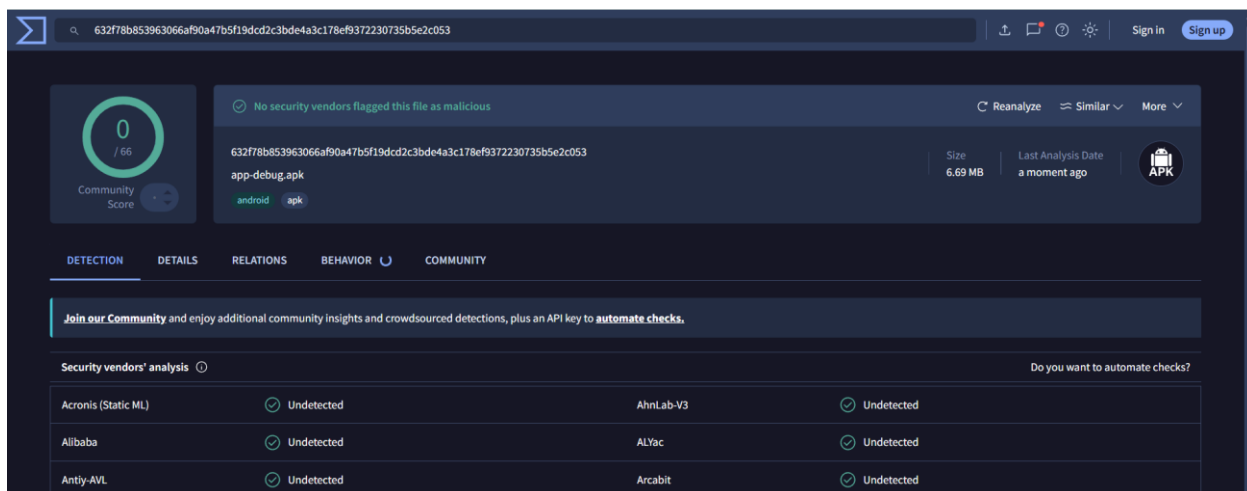


Рисунок 26 – Проверка файла `app-debug.apk`

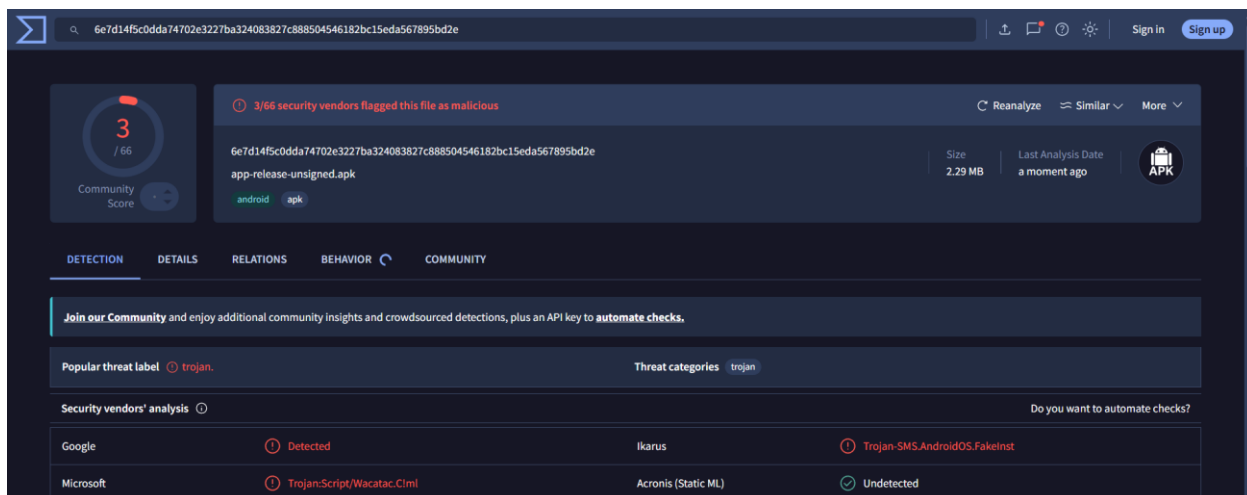


Рисунок 27 – Проверка файла `app-release-unsigned.apk`

Результат проверки VirusTotal apk-файлов на наличие вирусов представлены на рисунках 26 и 27. Программа показала, что три поставщика систем безопасности поместили файл с обфускацией как вредоносный и ни один не поместил файл без обфускации. Данное расхождение в результатах сканирования объясняется тем, что процесс обфускации кода намеренно изменяет структуру приложения, переименовывает классы и методы в нечитаемые обозначения, а также скрывает внутреннюю логику программы, что распространено среди разработчиков вредоносного ПО для уклонения от обнаружения. В связи с этим алгоритмы антивирусных систем автоматически классифицируют обфусцированный код как подозрительный, генерируя ложные срабатывания (false positives), тогда как отладочная версия приложения содержит читаемый код со

стандартными именами классов и методов, а также подписана отладочным сертификатом, что не вызывает подозрений у детекторов безопасности.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы были освоены процессы разработки Android-приложений на языке Kotlin и проведения комплексного анализа безопасности арк-файлов. Созданное приложение включает три экрана (отображение ФИО студента, получение данных из REST API и форма обратной связи с сохранением в SQLite базу данных). В результате анализа арк-файлов с помощью инструментов ArkTool, arkleaks и VirusTotal было продемонстрировано, что обфускация кода с использованием R8/ProGuard эффективно затрудняет статический анализ приложения, скрывая имена классов и методов, а также частично маскируя чувствительную информацию (URL, endpoints). Однако полное устранение утечек информации невозможно, так как некоторые данные должны оставаться доступными для функционирования приложения. Также было выявлено, что обфусцированный код может вызывать ложные срабатывания антивирусных систем, что является нормальной реакцией на запутанную структуру кода, характерную как для защищенных приложений, так и для вредоносного ПО.